

Week 12 : Deployment on AWS

This week, we will learn how to deploy our ThreadHive Full Stack application on AWS using Claude Code. The deployment process involves several steps, including setting up an EC2 instance, installing Docker, cloning the app code, building Docker images, and running containers. Below are the detailed steps along with Claude Code prompts to guide you through the process.

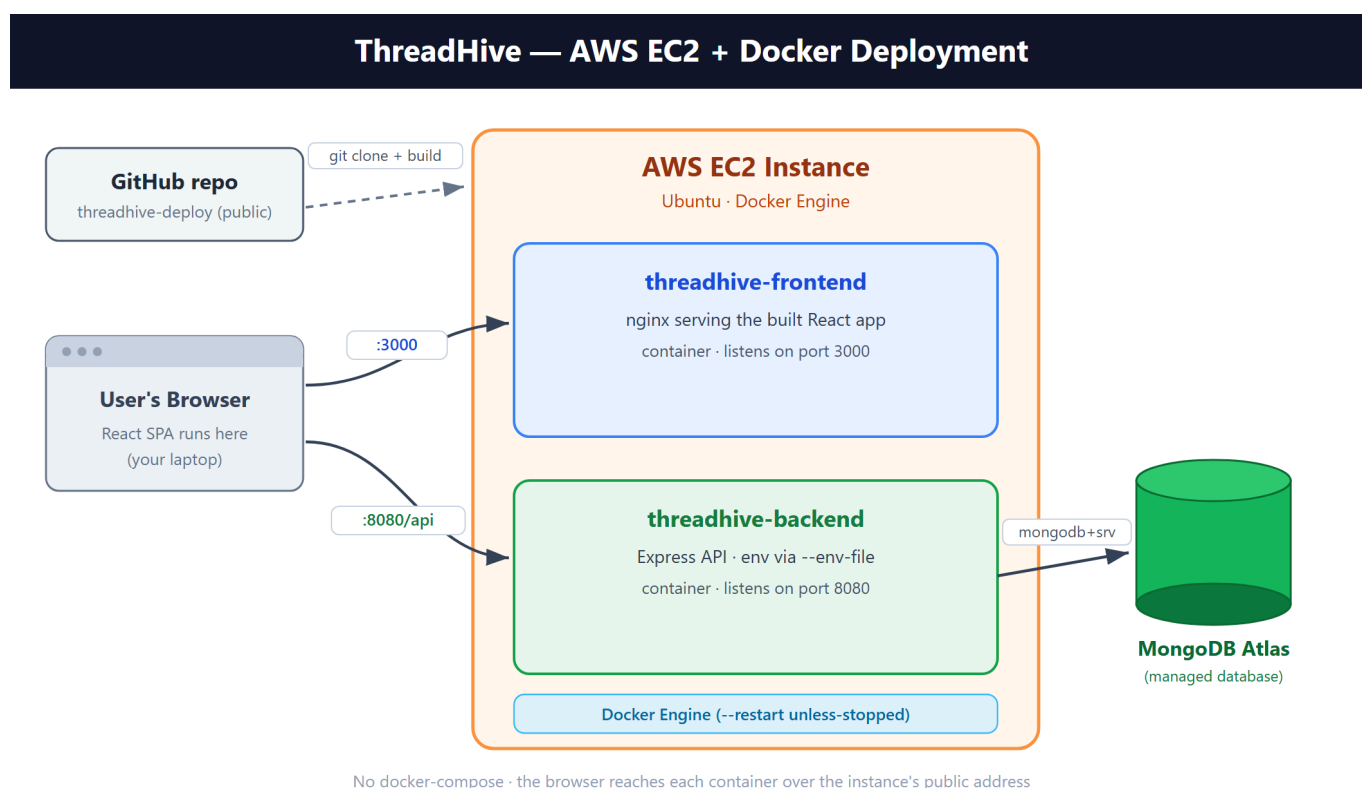
Every step includes a **Claude Code prompt** that can be used to execute that deployment step. Each step also has a  **[OPTIONAL] Doing it manually** subsection showing the underlying commands/files, so you understand what Claude does under the hood (and can do it by hand if needed).

Project Setup and Structure

This week's starter code is the ThreadHive codebase built last week.

1. Download and extract the assignment starter code from the zip file provided on Olympus, and open the **entire project folder** in VS Code so Claude Code can see both **threadhive-backend** and **threadhive-frontend**.
2. Open a VS Code terminal in the project root and launch Claude Code with the **claude** command.

Deployment Architecture



1. The frontend and backend applications are deployed as separate Docker containers on a single AWS EC2 instance.
2. The EC2 instance is running Ubuntu Linux.

3. The frontend is served by nginx on port 3000, while the backend API runs on port 8080.
4. The backend connects to a MongoDB Atlas cluster for data storage.
5. The codebase on the EC2 instance is cloned from a GitHub repository, built from the local codebase.

Note: nginx is a lightweight web server that serves the static files generated by the frontend build process. It also handles routing for single-page applications (SPAs) like React, ensuring that client-side routes work correctly.

Quick Start Execution Guide

#	Step	Where
1	Generate the two Dockerfiles	Local machine
2	Make the frontend API URL configurable	Local machine
3	Initialize git & push codebase to GitHub	Local machine
4	Launch an EC2 instance	AWS Console
5	Connect to EC2 & install Docker	Claude Code → EC2
6	Clone the repo onto EC2	Claude Code → EC2
7	Build the two Docker images	Claude Code → EC2
8	Create the backend <code>.env</code> on EC2	Claude Code → EC2
9	Run the two containers	Claude Code → EC2
10	Whitelist the EC2 IP in MongoDB Atlas	MongoDB Atlas
11	Access & verify the app	Browser

Prerequisites:

- An AWS account with EC2 access
- Claude Code installed and signed in
- A MongoDB Atlas cluster (connection string ready)
- The GitHub CLI (`gh`) authenticated locally (`gh auth status`) — Claude uses it to create the repo
- Your MERN app code, structured as a monorepo with `*-backend/` and `*-frontend/` folders

Step 1: Generate the Dockerfiles

A **Dockerfile** is a recipe that packages your app into a portable **image** — base OS, runtime, dependencies, and startup command — so it runs identically anywhere. For a MERN stack you write one per service. The frontend uses a **multi-stage build**: stage 1 compiles the React app with Vite, stage 2 copies the static output into a tiny **nginx** image to serve it.

In the Claude Code CLI (from the project root), enter the following prompt:

```
Help me create Dockerfiles for the frontend and backend apps. Frontend will
run
on port 3000 and backend on 8080. Keep the docker files separate and do not
use
docker compose. Use nginx to serve the frontend files. For the backend, do
not
copy the .env file into the Docker image: the env variables will be
injected
using a env file separately.
```

This should create the following files:

- `threadhive-backend/Dockerfile` — will install production deps only, will run the server on 8080.
- `threadhive-frontend/Dockerfile` — multi-stage (Vite build → nginx), will serve on 3000.
- `threadhive-frontend/nginx.conf` — listens on 3000 with an SPA fallback (`try_files ... /index.html`) so client-side routes work.
- A `.dockerignore` in each folder (keeps `node_modules` and any `.env` out of the image).

Why `.env` is excluded: secrets must never be baked into an image. They are injected at run time with `docker run --env-file` (Step 9).

[OPTIONAL] Doing it manually

If not using Claude, create these files in your project. Note that there may be slight variations in the Dockerfiles generated by Claude.

`threadhive-backend/Dockerfile`:

```
# syntax=docker/dockerfile:1
FROM node:26-alpine

WORKDIR /app

# The app reads process.env.PORT; default it to 8080 (overridable at run
time).
ENV NODE_ENV=production \
    PORT=8080

# Install production dependencies first for better layer caching.
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Copy the rest of the application source.
```

```
COPY . .

# Run as the unprivileged user that ships with the node image.
USER node

EXPOSE 8080
CMD ["node", "main.js"]
```

threadhive-backend/.dockerignore:

```
node_modules
npm-debug.log
.env
.env.*
tests
coverage
.git
.gitignore
*.md
Dockerfile
.dockerignore
```

threadhive-frontend/Dockerfile (the API-URL build arg is added in Step 2):

```
# syntax=docker/dockerfile:1

# ---- Build stage: compile the React + Vite SPA ----
FROM node:26-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Serve stage: static files via nginx on port 3000 ----
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 3000
CMD ["nginx", "-g", "daemon off;"]
```

threadhive-frontend/nginx.conf:

```
server {
    listen 3000;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # SPA: let react-router handle client-side routes.
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Vite produces content-hashed assets, so cache them hard.
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

threadhive-frontend/.dockerignore:

```
node_modules
dist
.env
.env.*
.git
.gitignore
coverage
*.md
Dockerfile
.dockerignore
```

Step 2: Make the frontend API URL configurable

Our frontend codebase hardcodes the backend URL (ThreadHive had `http://localhost:5000/api` in `axiosInstance.js`). That only works during local development. Once deployed, the browser must call the **EC2 public address**, so the URL has to be configurable. With Vite, you expose it as a `VITE_`-prefixed env var that gets **inlined at build time**, and you pass it into the Docker build with `--build-arg`.

Claude code might already prompt you to make this change in Step 1, but if not, you can do it manually:

make the baseURL in the frontend configurable with VITE_API_BASE_URL.

This should change `axiosInstance.js` to read from `import.meta.env.VITE_API_BASE_URL` (with a localhost fallback for dev). The frontend `Dockerfile` gains `ARG VITE_API_BASE_URL → ENV VITE_API_BASE_URL` before `npm run build`. Claude might modify the `.env.example` to document the variable.

Note: because Vite inlines `VITE_API_BASE_URL` during `npm run build`, changing the API URL later means **rebuilding** the frontend image — you can't override it with `docker run`.

🔧 [OPTIONAL] Doing it manually

1. Read the env var in `threadhive-frontend/src/api/axiosInstance.js`:

```
import axios from "axios";

// Configurable via VITE_API_BASE_URL (inlined at build time by Vite).
// Falls back to the local dev backend when unset.
const baseURL = import.meta.env.VITE_API_BASE_URL ||
"http://localhost:5000/api";

const axiosInstance = axios.create({ baseURL });

export default axiosInstance;
```

2. Accept it as a build arg in `threadhive-frontend/Dockerfile` — add these two lines in the build stage, right after `WORKDIR /app`:

```
ARG VITE_API_BASE_URL
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
```

3. Document it in `threadhive-frontend/.env.example`:

```
# Inlined into the build by Vite (read at build time, not run time).
VITE_API_BASE_URL=http://localhost:5000/api
```

Step 3: Initialize git & push to GitHub

The AWS EC2 instance gets your code by cloning it from GitHub, so the repo (including the Dockerfiles from Steps 1–2) must be pushed first. Prompt Claude to do this as follows:

```
Initialize a git repo, commit the current version and push the changes to a
remote repo https://github.com/<your-username>/<your-repo>
```

Claude uses the authenticated **gh** CLI to create the remote repository and push to it.

Public vs private: Claude might ask which visibility you want. We will clone the repo over plain HTTPS later, which is simplest with a **public** repo. If you keep it **private**, you'll need a GitHub Personal Access Token or a deploy key on the instance to clone.

The above prompt should initialize a git repository, create a **.gitignore** (that excludes **node_modules/**, **.env**, build output, etc.), make an initial commit, create the GitHub repo via **gh**, and push **main**.

Set up the GitHub CLI [ONLY IF NOT DONE PREVIOUSLY]

We had set up the **gh** CLI previously. If not set up, follow these steps:

1. Install the GitHub CLI from cli.github.com (or **winget install GitHub.cli** on Windows / **brew install gh** on macOS).
2. Authenticate:

```
gh auth login
```

Choose **GitHub.com** → **HTTPS** → **authenticate via browser**.

3. Verify:

```
gh auth status
```

[OPTIONAL] Doing it manually

1. Create a **.gitignore** at the repo root so secrets and dependencies aren't committed:

```
node_modules/
.env
.env.*
!.env.example
dist/
build/
```

```
*.log
coverage/
```

2. Initialize, commit, and push:

```
git init -b main
git add -A
git commit -m "Initial commit: app + Docker setup"

# Create the GitHub repo and push in one go (requires the gh CLI,
authenticated):
gh repo create <your-username>/<your-repo> --public --source=. --
remote=origin --push
```

Or, **without gh** — create an empty repo in the GitHub web UI first, then:

```
git remote add origin https://github.com/<your-username>/<your-repo>.git
git push -u origin main
```

Step 4: Launch an EC2 Instance [Manual Setup in AWS Console]

An EC2 instance is a virtual server in the cloud that will run your containers. You choose the OS, size, and network rules (the **security group**, which is a firewall).

1. AWS Console → EC2 → Launch Instance

2. Configure:

- **Name:** `threadhive-app`
- **OS:** Ubuntu (e.g. 24.04 LTS or 26.04 LTS)
- **Instance type:** `t3.small` or `t3.micro` (free tier)
- **Key pair:** create one (e.g. `my-key`) and **download the .pem** and save it in `~/.ssh/` folder.

3. Network settings → Security group — add inbound rules:

Type	Port	Source	Purpose
SSH	22	My IP, or 0.0.0.0/0	Your SSH access
Custom TCP	3000	Anywhere (0.0.0.0/0)	Frontend (nginx)
Custom TCP	8080	Anywhere (0.0.0.0/0)	Backend API

4. **Launch**, then copy the instance's **Public IPv4 DNS** (e.g. `ec2-54-167-86-232.compute-1.amazonaws.com`) — you'll use it in the remaining steps.

⚠ The ports **must** match your app: `3000` (frontend) and `8080` (backend). If they don't, the browser can't reach the containers even though they're running.

Note: Though the creation of AWS EC2 instance can be automated with tools like Terraform or AWS CLI, this is beyond the scope of this exercise.

Step 5: Connect to EC2 & Install Docker

SSH is an encrypted protocol that lets Claude run commands on your server using the `.pem` key as the credential. Your fresh Ubuntu box has no Docker yet, so we install the latest **Docker Engine** from Docker's official apt repository.

Prompt Claude to connect and install Docker:

```
Connect to the EC2 instance <ec2-public-dns> and install the latest version
of
docker on the instance. The ssh key is located in @~/.ssh/<key-file.pem>
```

For ex, sample values might be: host `ec2-54-167-86-232.compute-1.amazonaws.com`, key `C:\Users\<you>\.ssh\my-key.pem.*`

Claude should SSH into the instance, then run the commands to install Docker from the official Docker repository. It might also add your user to the `docker` group so you can run `docker` without `sudo`.

Default SSH user: On Ubuntu AMIs the login user is `ubuntu` (not `ec2-user`, which is for Amazon Linux). If you get `Permission denied (publickey)`, the username is usually the culprit.

Group membership: adding `ubuntu` to the `docker` group only takes effect in a **new** session, so reconnect (or use `sudo docker`) before running Docker without `sudo`.

🔧 [OPTIONAL] Doing it manually

1. Connect (from a terminal in your local machine):

```
# macOS/Linux: lock down the key first, or SSH will refuse it
chmod 400 my-key.pem
ssh -i my-key.pem ubuntu@<ec2-public-dns>
```

On **Windows**, `chmod` doesn't apply; the key in your user profile usually works as-is.

2. On the instance — install Docker from the official apt repo:

```
# Prereqs
sudo apt-get update
sudo apt-get install -y ca-certificates curl

# Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository (auto-detects your Ubuntu codename)
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

# Install Docker Engine + plugins
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin

# Enable, start, and grant your user access
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
newgrp docker          # or log out and back in

# Verify
docker --version
docker run --rm hello-world
```

Step 6: Clone the repo onto EC2

The instance needs your source code (and the Dockerfiles) to build images. Cloning from GitHub keeps the workflow version-controlled — push changes locally, then `git pull` and rebuild on EC2.

Prompt Claude to clone the repo as follows:

Clone the github repo `https://github.com/<your-username>/<your-repo>` on the EC2 instance.

Claude will connect to your EC2 instance and clone the repo to `~/<your-repo>` (e.g. `/home/ubuntu/threadhive-deploy`), containing the `*-backend/` and `*-frontend/` folders.

[OPTIONAL] Doing it manually

```
# On the EC2 instance (git ships with Ubuntu; if missing: sudo apt-get
install -y git)
git clone https://github.com/<your-username>/<your-repo>.git
cd <your-repo>
ls          # you should see threadhive-backend/ and threadhive-frontend/
```

Step 7: Build the two Docker images

`docker build` reads each `Dockerfile` and produces an immutable image on the instance. The **frontend build needs the API URL** (from Step 2) passed as a build argument, because Vite bakes it into the bundle right now.

Prompt Claude to build the images:

Build the two docker images on the EC2 instance.

Claude might ask **what API base URL to bake into the frontend**. Answer with your instance's public address + backend port + `/api`:

`http://<ec2-public-dns>:8080/api`

(e.g. `http://ec2-54-167-86-232.compute-1.amazonaws.com:8080/api`)

This should create two images: `threadhive-backend:latest` and `threadhive-frontend:latest`. The backend image is a Node environment with your server code; the frontend image is nginx serving the compiled React app.

 Get this URL right — it's compiled in. A wrong value (e.g. `localhost`) means the deployed site loads but every API call fails. Fixing it requires rebuilding the frontend image.

[OPTIONAL] Doing it manually

```
# On the EC2 instance, from the repo root (e.g. ~/threadhive-deploy):

# Backend — no build args needed
docker build -t threadhive-backend:latest ./threadhive-backend
```

```
# Frontend – bake in the public API URL
docker build \
  --build-arg VITE_API_BASE_URL=http://<ec2-public-dns>:8080/api \
  -t threadhive-frontend:latest \
  ./threadhive-frontend

# Confirm
docker images | grep threadhive
```

Step 8: Create the backend `.env` on EC2

The backend's `.env` is gitignored, so it was **not** cloned. You must create it on the instance. It supplies the database connection and secrets that get injected at run time. Set up the `.env` file locally with the correct environment variables and then ask Claude to copy it to the EC2 instance using `scp`:

Use `scp` to copy the `.env` file from the backend folder on my local machine to the EC2 instance.

[OPTIONAL] Doing it manually

Either create the file directly on the instance:

```
# On the EC2 instance
cat > ~/<your-repo>/threadhive-backend/.env <<'EOF'
PORT=8080
MONGODB_URI=mongodb+srv://<user>:
<password>@<cluster>.mongodb.net/threadhive
JWT_SECRET=your-strong-secret
GEMINI_API_KEY=your-gemini-key
GEMINI_MODEL=gemini-2.5-flash
EOF
```

...or copy your local `.env` up with `scp` (run from your **local** machine):

```
scp -i my-key.pem ./threadhive-backend/.env \
  ubuntu@<ec2-public-dns>:~/<your-repo>/threadhive-backend/.env
```

Step 9: Run the two containers

An image is a blueprint; a **container** is a running instance of it. We run both **detached** (**-d**) so they survive your SSH session closing, with **--restart unless-stopped** so they come back after a reboot. The backend gets its environment from **--env-file**.

Prompt Claude to run the containers:

```
Run the two containers on the EC2 instance. I have created a .env file at
~/<your-repo>/threadhive-backend/.env and configured the security group.
Run the backend on port 8080 loading variables with --env-file, and the
frontend on port 3000. Use --restart unless-stopped for both containers.
```

This should run both containers on the EC2 instance. A **docker ps** would show both containers **Up**. The backend logs should show "MongoDB connected" if the connection string is correct and the EC2 IP is whitelisted in Atlas.

[OPTIONAL] Doing it manually

```
# On the EC2 instance:

# Backend - inject env from the file; force PORT=8080 to match the mapping
docker run -d --name threadhive-backend --restart unless-stopped \
  --env-file ~/threadhive-deploy/threadhive-backend/.env \
  -e PORT=8080 -p 8080:8080 \
  threadhive-backend:latest

# Frontend
docker run -d --name threadhive-frontend --restart unless-stopped \
  -p 3000:3000 \
  threadhive-frontend:latest

# Verify
docker ps
docker logs threadhive-backend      # look for "MongoDB connected"
```

Re-deploying? Remove the old container first: **docker rm -f threadhive-backend** (and **threadhive-frontend**) before re-running, or you'll get a name conflict.

Step 10: Whitelist the EC2 IP in MongoDB Atlas (Manual Setup in Atlas UI)

Atlas blocks all connections except from IPs on its **Network Access** list. Your backend's outbound calls will be rejected until the EC2 public IP is allowed.

Steps in the Atlas UI: **Security** → **Network Access** → **Add IP Address** → enter your EC2 public IP (or **0.0.0.0/0** for testing only) → name it "EC2 Instance" → Confirm.

Note: If the backend logs show a Mongo connection/timeout error and the container keeps restarting, this is the most common cause.

Step 11: Access & verify your app

In a browser on your local machine, open the deployed ThreadHive app: `http://<ec2-public-dns>:3000`

Test out the application. You should be able to register, log in, and see threads. If environment variables are correctly set up, should be see all the features of the app working, including the AI features and the search and bookmarking features.

Important Notes

1. **The frontend API URL is fixed at build time.** To change it (or after the public IP changes), rebuild the frontend image with a new `--build-arg VITE_API_BASE_URL=...`
2. **Public IP/DNS changes on stop/start.** Attach an **Elastic IP** to keep it stable, otherwise update and rebuild after each restart.
3. **Never commit secrets.** Keep `.pem` keys and `.env` out of git (the generated `.gitignore` already excludes `.env`).
4. **No docker-compose here.** The two containers are independent and talk only through the browser; there's no shared Docker network to configure.
5. **0.0.0.0/0 opens ports to the whole internet.** Fine for a class exercise; lock it down for anything real.
6. **Monitor with** `docker ps`, `docker logs <name>`, and `docker stats`.